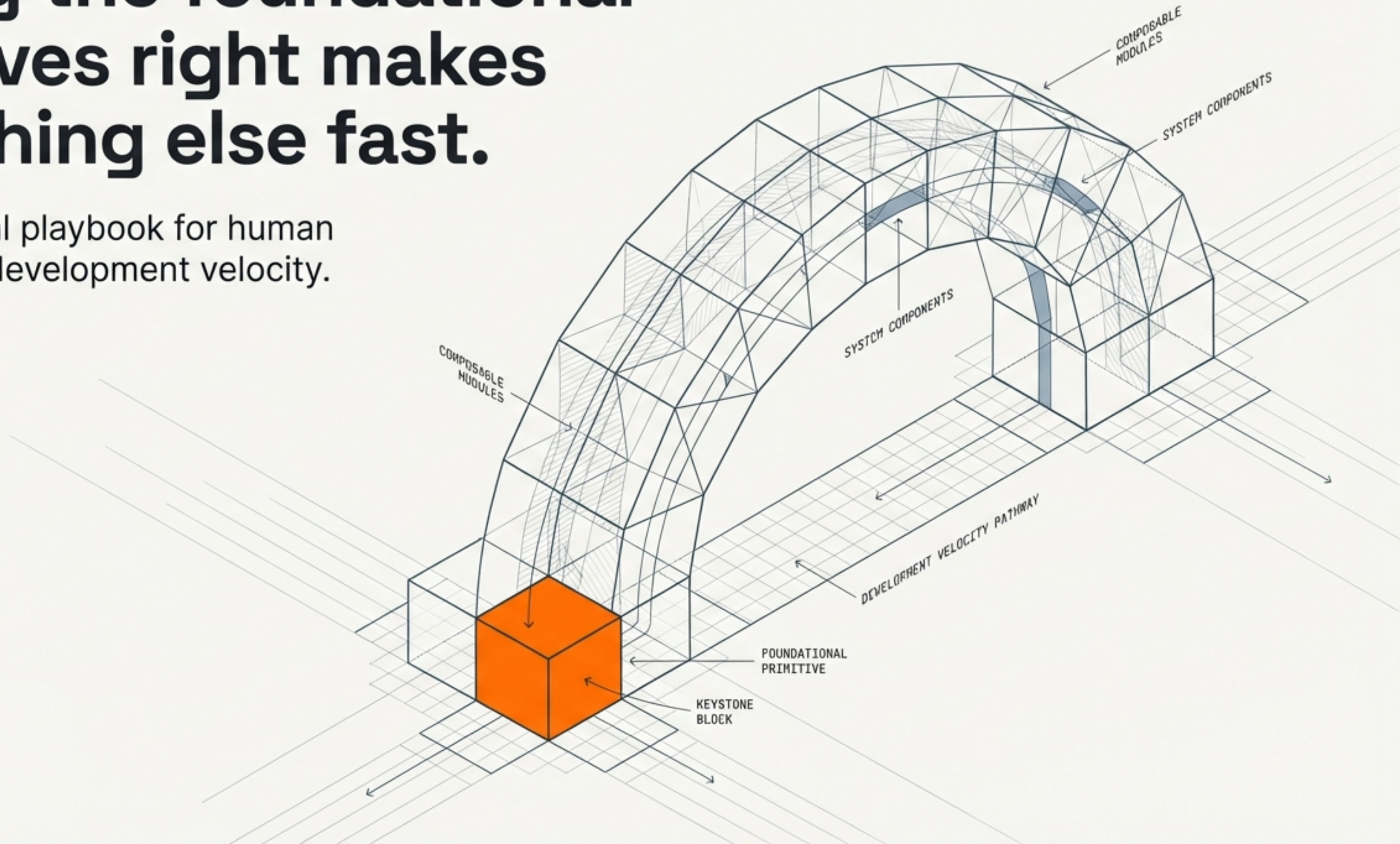


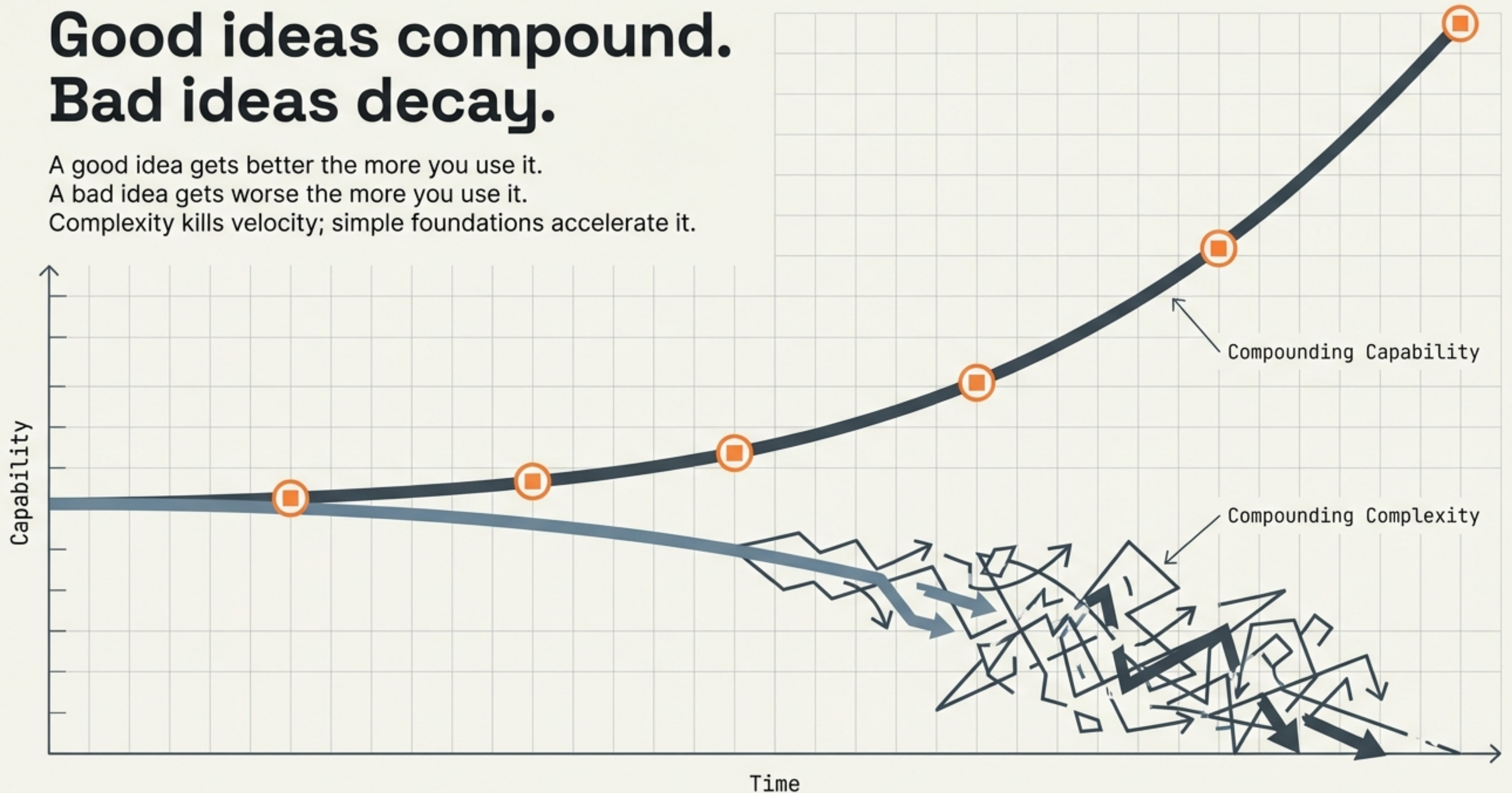
Getting the foundational primitives right makes everything else fast.

An architectural playbook for human and AI-driven development velocity.



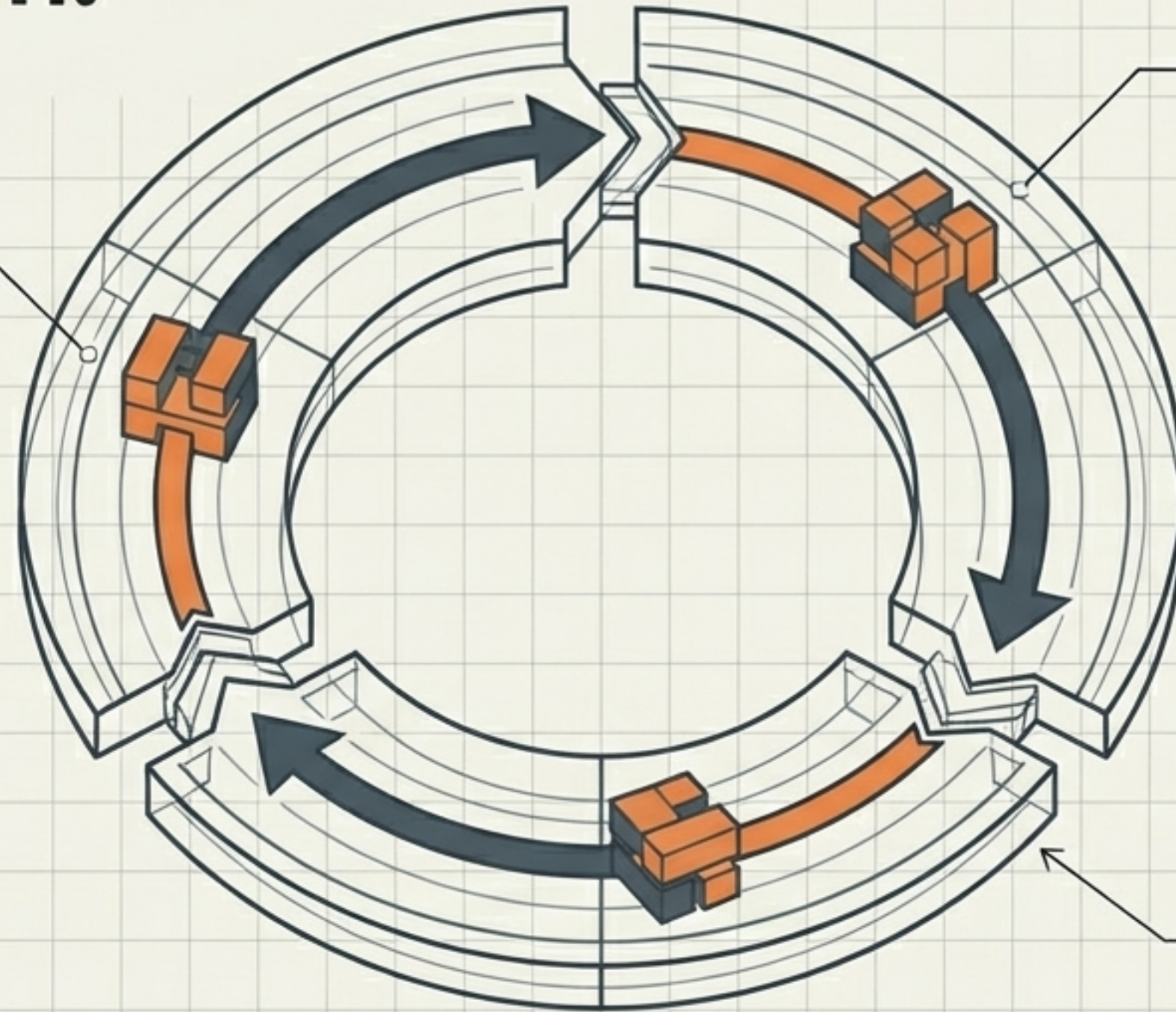
Good ideas compound. Bad ideas decay.

A good idea gets better the more you use it.
A bad idea gets worse the more you use it.
Complexity kills velocity; simple foundations accelerate it.



The evolutionary engine of production.

Commoditise:
Strip away the bespoke elements until it becomes an invisible, rock-solid primitive.



Innovate:
Discover what works.
Unpredictable, high-speed iteration.

Leverage:
Extract the successful patterns. Apply them across multiple contexts.

Structuring 18+ AI agents across three phases of code.

Explorers



Objective: **Prototype** fast, discover what works.

Code Relationship: Builds fast and messy. Ignores sunk costs.

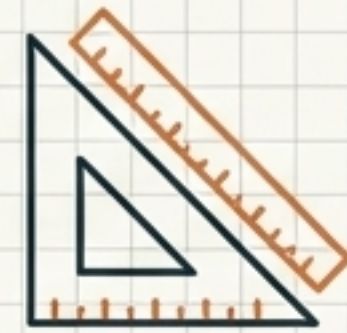
Villagers



Objective: **Productise**, test, harden.

Code Relationship: **Cleans** up, tests, and prepares the code for broader use.

Town Planners



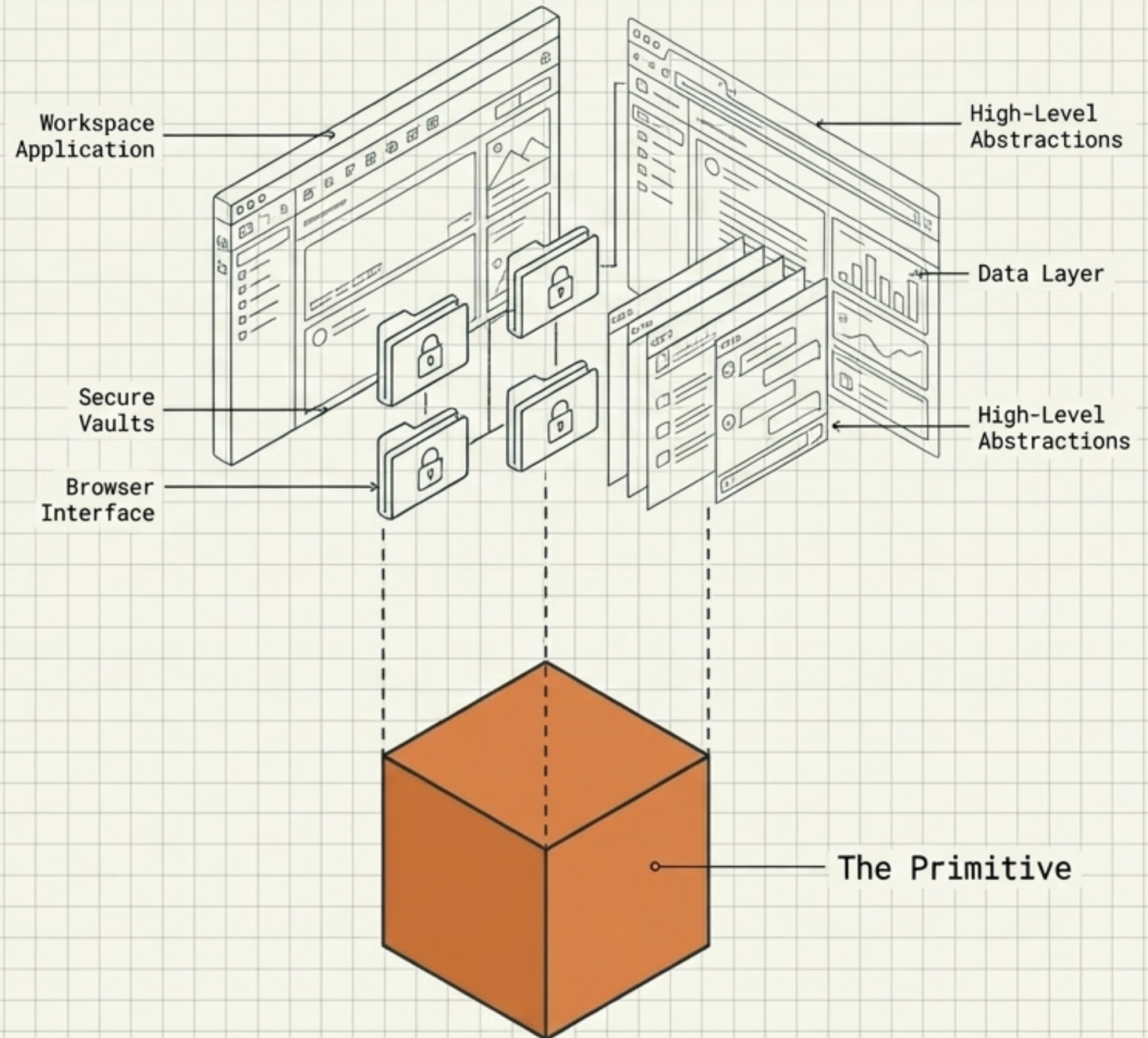
Objective: **Commoditise**, optimise, scale.

Code Relationship: Turns proven patterns into **invisible**, flawless primitives.

The anatomy of a perfect primitive.

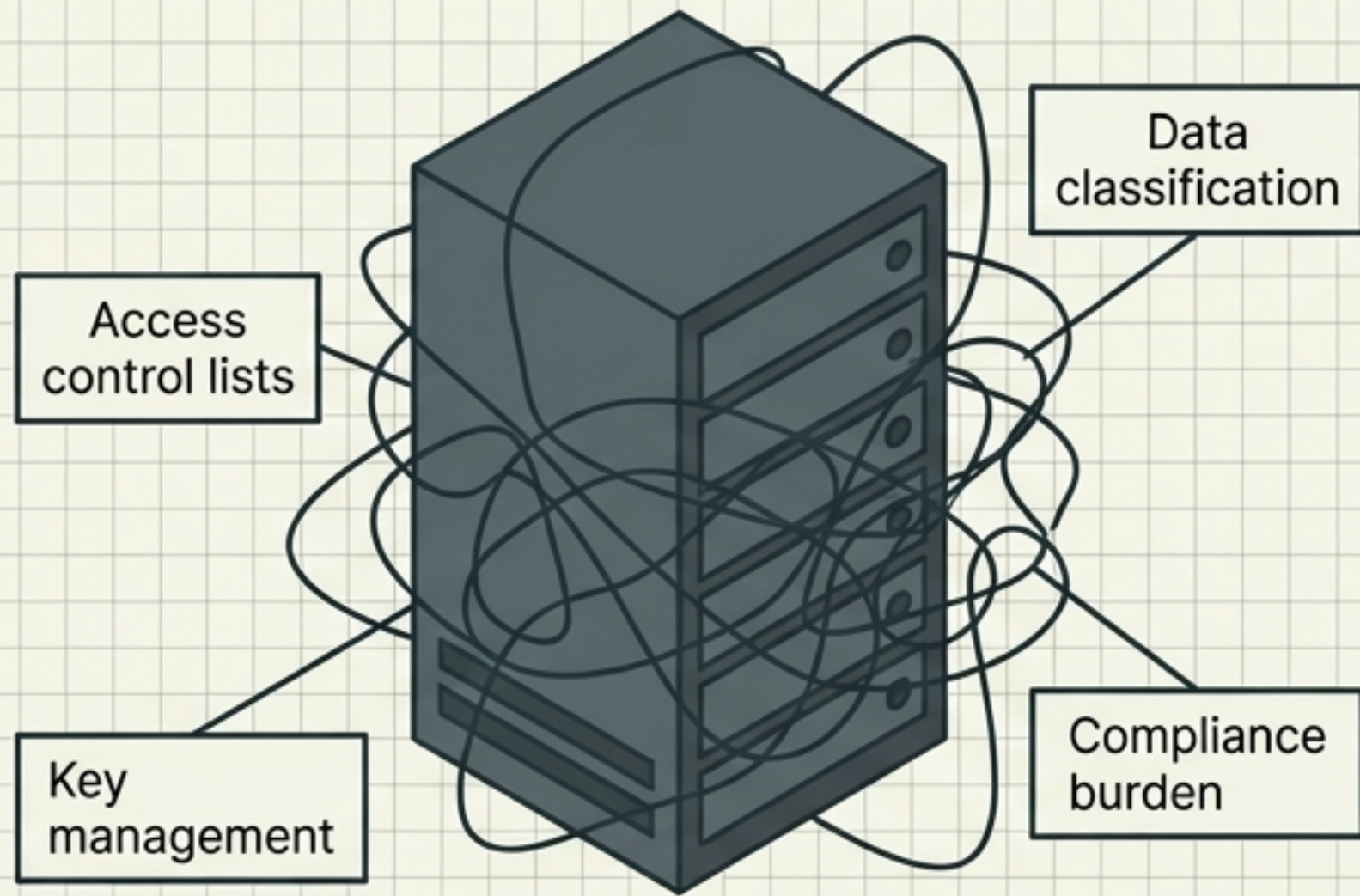
A primitive is a building block so simple that it becomes invisible. You stop thinking about it and start building on top of it.

Case Study: Encrypt in the browser, send the ciphertext, the key stays with the user. It does one thing perfectly and adds zero complexity to the layers built above it.

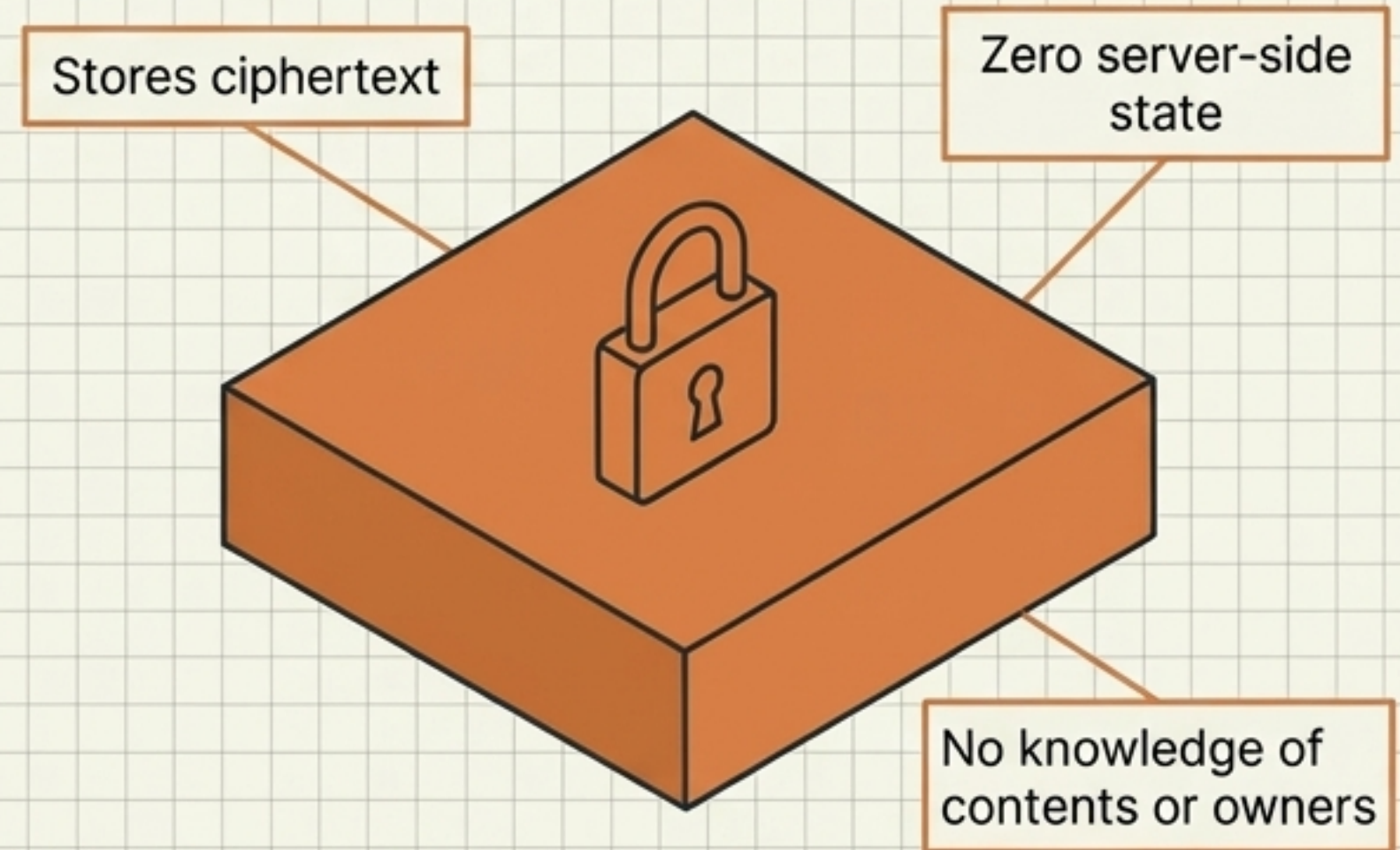


The velocity of a blind server architecture.

By pushing state and encryption to the client, a massive category of backend complexity is entirely eliminated.

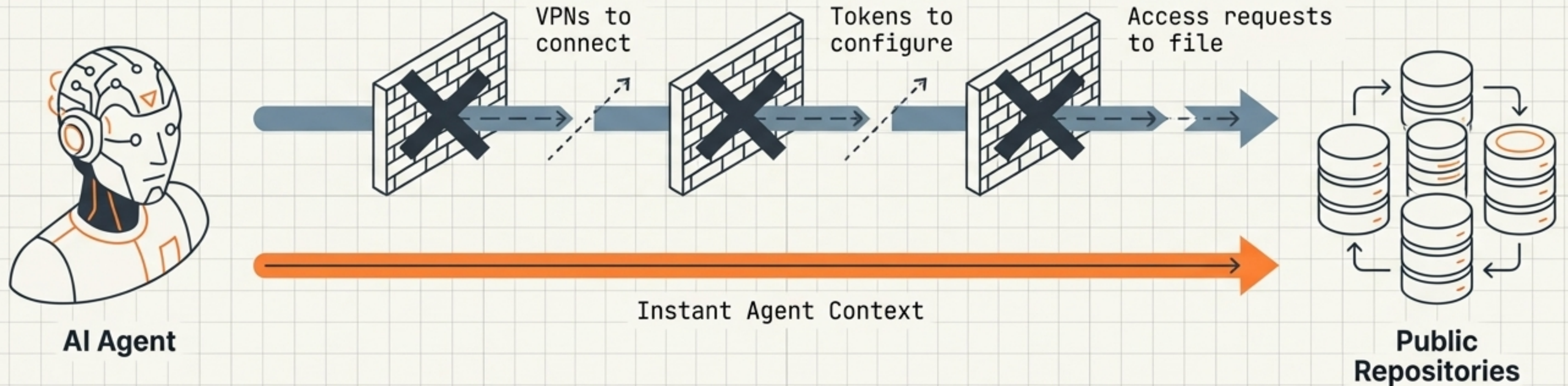


Traditional Architecture



Primitive-Driven Architecture

Open source as a critical velocity lever.



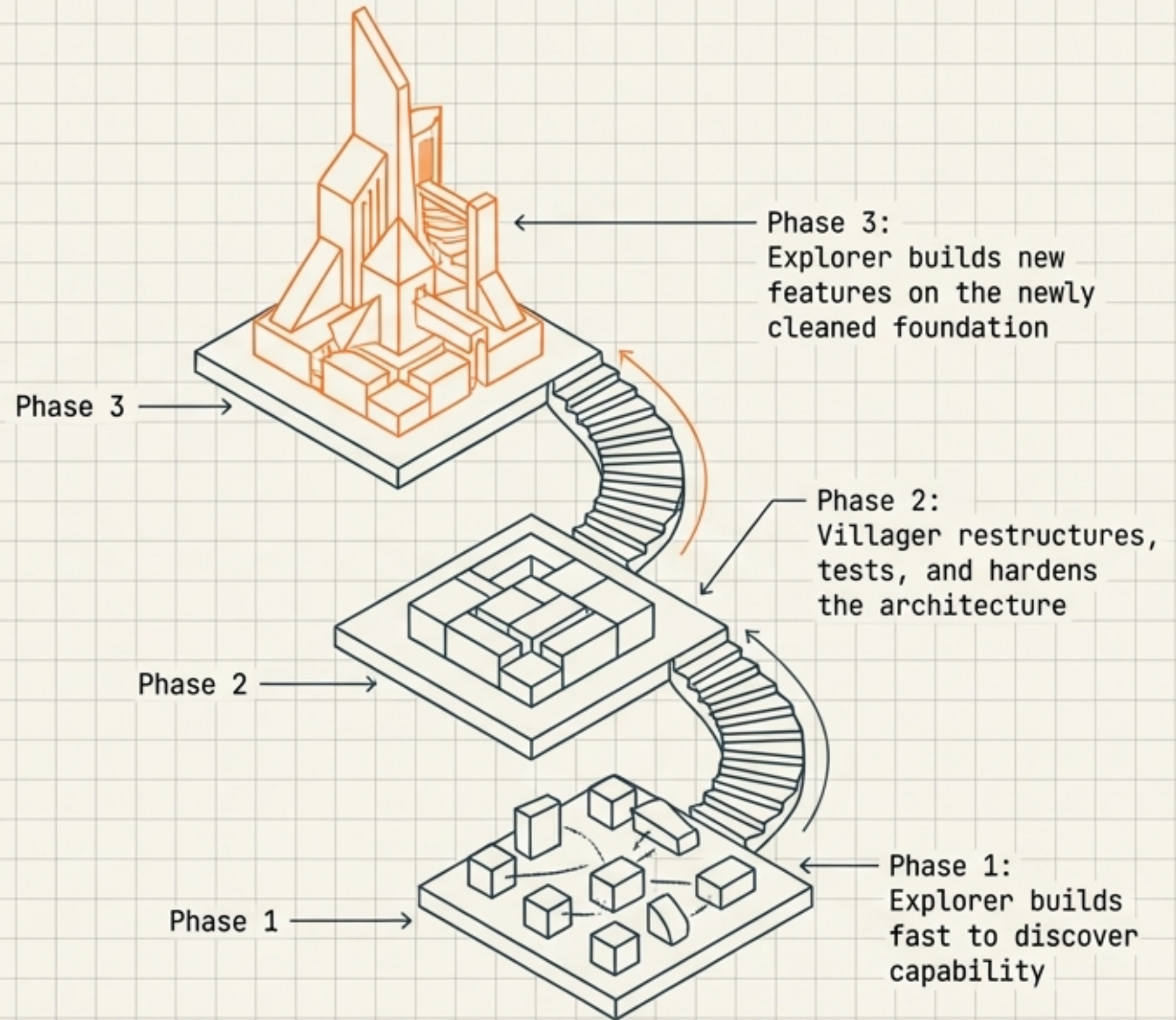
1. Public Repositories: Components are instantly pullable.

2. Zero Access Friction: No authentication hurdles block the workflow.

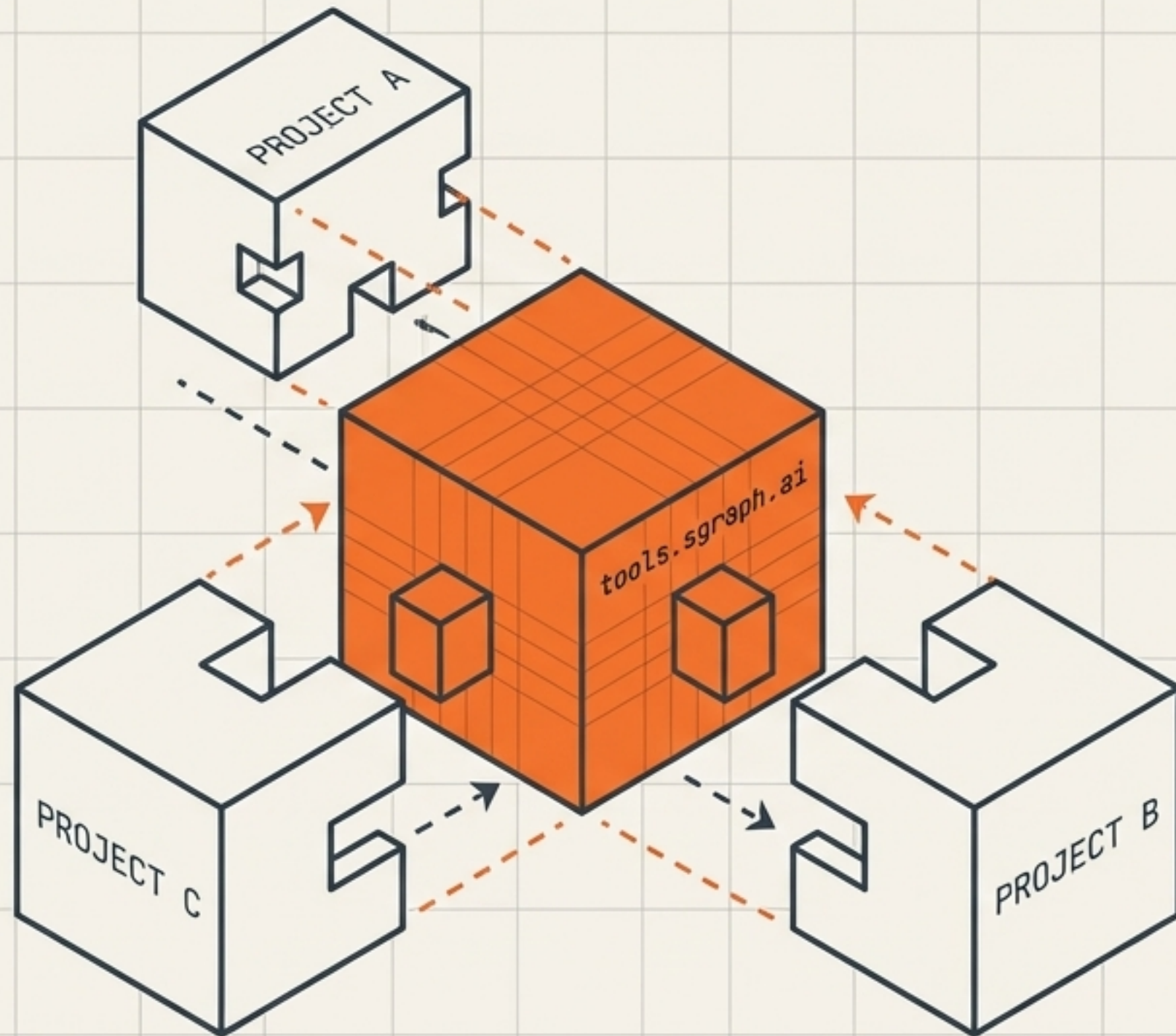
3. Instant Context: Agents seamlessly pull repos, read code, and produce briefs spanning multiple codebases.

Constant refactoring and the rejection of sunk costs.

Never protect past code. Proprietary lock-in and sunk cost fallacies are active threats to velocity.



Frictionless component composability.



Technology:	Web Components with Shadow DOM.
Philosophy:	Components do one thing well and do not depend on each other's internals.
Architecture:	Dependency inversion pattern.
Result:	Drop a component into a new context, and it instantly works.

Why AI agents thrive in this exact architecture.

The Primitives

**Simple
Architecture**

**Open
Codebases**

**Zero Server
State**

**Independent
Components**

Agent Capability

Understandable
from a single
context brief

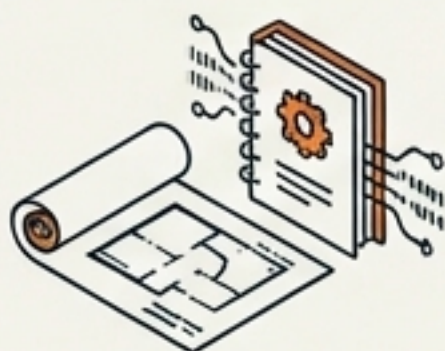
Instantly
readable without
authentication
blocking

Features
implemented
entirely on the
client side

Isolated
changes,
reducing
hallucination
risk

The agentic workflow in action: Building a branch model

1

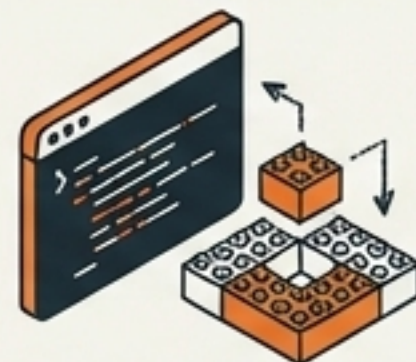


The Architect Agent

Reads the **CLI code**, the **send server code**, and the **vault spec** in a single session to design the model.



2



The Developer Agent

Implements the branch model **entirely client-side**, requiring zero updates to the **backend**.



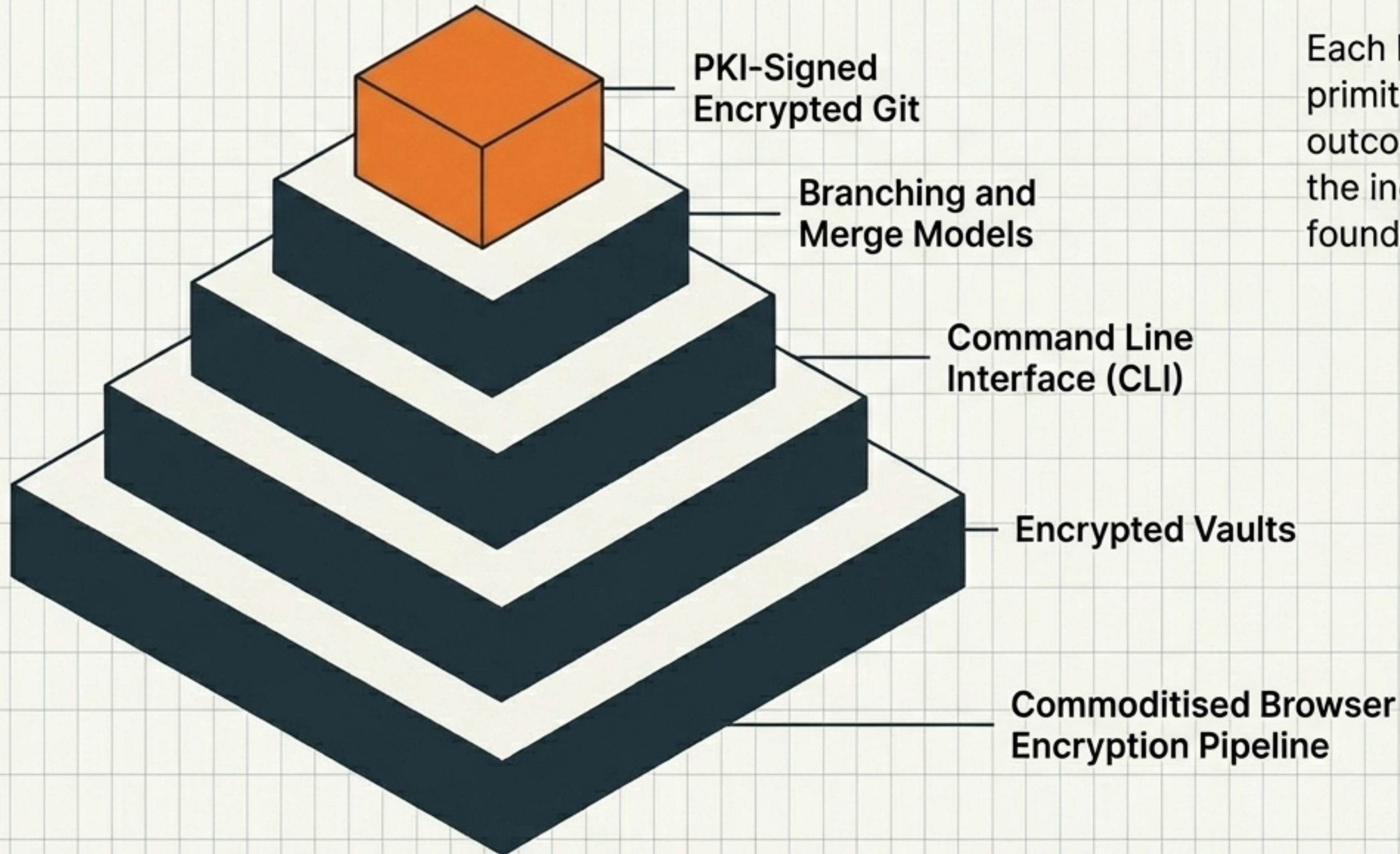
3



The QA Agent

Pulls the **repo** and spins up a **local server instance** in seconds for integration testing.

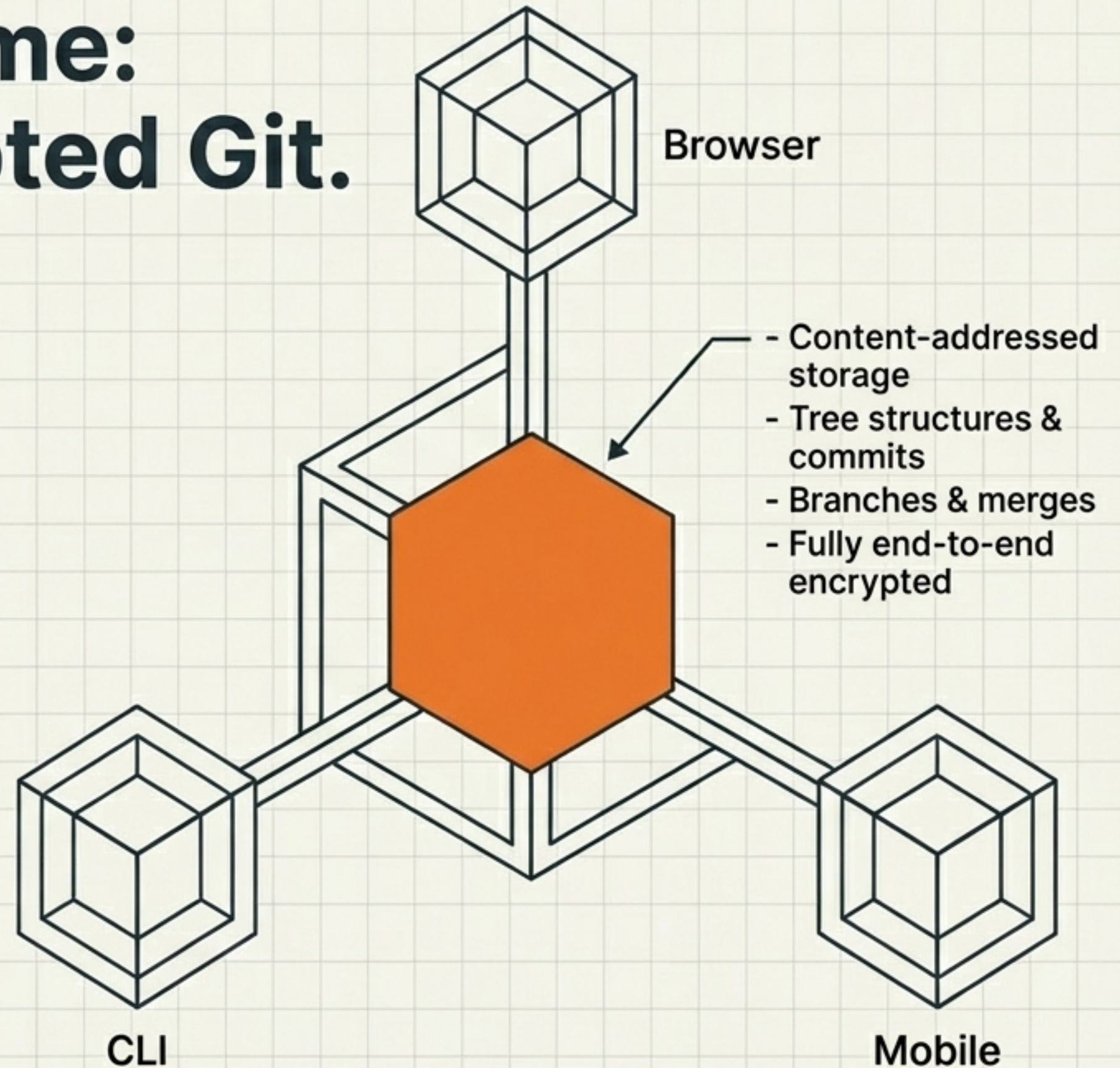
The compounding capability stack.



Each layer became a pristine primitive for the next. The outcome was not planned; it was the inevitable result of flawless foundations.

The emergent outcome: A PKI-signed, encrypted Git.

This solves multi-user collaboration for documents with zero backend state. It felt small to build because the foundations did the heavy lifting.



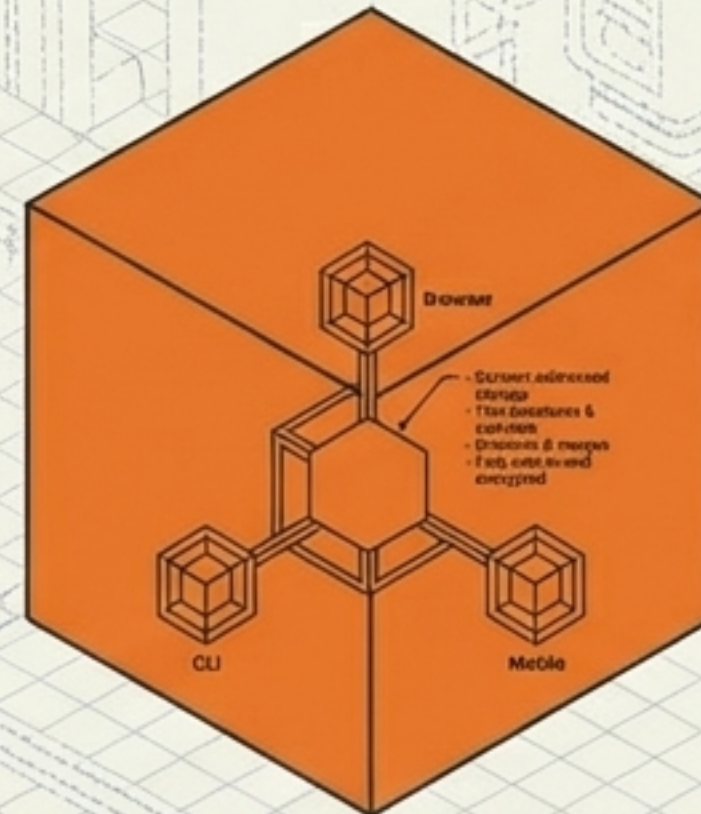
The emergent outcome is just the current layer.

The PKI-signed encrypted Git is not a destination. It has already been commoditised. The Explorers are already building the next unknown capability on top of it.

UNKNOWN
NEXT
CAPABILITY

FUTURE
PROTOCOL
LAYERS

UNEXPLORED
FRONTIERS



**When the primitives are
right, the hard things start
to feel approachable.**

